PRÁCTICA 04

Análisis de empleados con Pandas

Del archivo Excel a los gráficos avanzados

Tema	Análisis exploratorio, agregaciones y visualización
Lenguaje	Python 3.14
Dependencias	pandas, openpyxl, matplotlib, seaborn, numpy
Dataset	500 empleados ficticios · 4 variables
Fecha	Octubre de 2026
Entrega	Código, Excel de resultados, gráficos y documento

Documento de soluciones con explicación, código fuente y evidencia de ejecución.

Contenido

- 1. Contexto y objetivos
- 2. Dataset de trabajo
- 3. Estructura del proyecto
- 4. Metodología del análisis
- 5. Código y explicación
- 6. Resultados del análisis
- 7. Visualizaciones
- 8. Conclusiones y respuestas
- 9. Ejecución en Visual Studio y VS Code
- Anexo A. Código completo
- Anexo B. Salida completa de consola

1. Contexto y objetivos

Una empresa ficticia posee un archivo de empleados con nombre, apellidos, edad y salario mensual. El objetivo es construir un análisis exploratorio con Python y Pandas para responder preguntas sobre el nivel salarial, la dispersión, los grupos de edad y la relación entre edad y salario.

Elemento	Detalle
Dataset base	datos_ficticios.xlsx · hoja Empleados
Registros	500 personas ficticias
Columnas	Nombre, Apellidos, Edad, Salario ($)
Nivel	Introductorio a intermedio con cierre avanzado
Duración sugerida	2 a 3 horas de laboratorio
Producto	resumen_empleados.xlsx y seis gráficos PNG

Preguntas que guían el análisis: ¿cuál es el salario promedio?, ¿qué tan dispersos están los salarios?, ¿qué grupos de edad concentran mejores salarios?, ¿existe relación entre edad y salario?, ¿cómo se comunican los resultados mediante gráficos?

Objetivo	Resultado esperado
Cargar el archivo Excel	DataFrame validado con 500 filas y 4 columnas.
Inspeccionar y limpiar	Columnas normalizadas y tipos de dato correctos.
Transformar variables	nombre_completo, grupo_edad y nivel_salarial.
Analizar con Pandas	groupby, agg, pivot_table y correlación.
Exportar resultados	Excel con tres hojas de salida.
Visualizar	Histograma, dispersión, barras, boxplot, heatmap y dashboard.

2. Dataset de trabajo

El archivo contiene 500 registros generados de forma reproducible. La edad se distribuye entre 18 y 70 años y el salario mensual se expresa en dólares. La tabla siguiente muestra las primeras ocho filas del archivo original.

VISTA PREVIA DEL DATASET

Nombre	Apellidos	Edad	Salario ($)
Albert	Martinez Lopez	50	5,120
Albert	Chavez Salazar	32	2,590
Sandra	Rojas Medina	57	2,940
Fernando	Jimenez Vega	64	4,200
Monica	Castillo Ramos	49	3,900
Darius	Russell Harper	57	3,210
Roger	Tyler Best	36	3,560
Patricia	Chavez Salazar	61	3,710

Columna	Tipo esperado	Descripción
Nombre	Texto	Nombre de la persona ficticia.
Apellidos	Texto	Apellidos de la persona ficticia.
Edad	Entero	Edad entre 18 y 70 años.
Salario ($)	Entero	Salario mensual en dólares.

La generación utiliza una semilla fija para que los resultados sean reproducibles. El salario se construye con una base asociada a la edad y un componente aleatorio, lo que produce una relación positiva que el análisis puede detectar.

3. Estructura del proyecto

La carpeta contiene el dataset, dos programas, el archivo de dependencias y las salidas generadas. El análisis puede ejecutarse varias veces sin perder los datos originales.

```
04_Pandas_Excel_Graficos/
|-- datos_ficticios.xlsx
|-- generar_datos.py
|-- analisis_empleados.py
|-- requirements.txt
|-- Solucion_Pandas_Excel_Graficos.docx
`-- salidas/
    |-- resumen_empleados.xlsx
    `-- graficos/
        |-- histograma_salarios.png
        |-- edad_vs_salario.png
        |-- promedio_grupo_edad.png
        |-- boxplot_salarios.png
        |-- correlacion.png
        `-- dashboard_empleados.png
```

Archivo	Función
generar_datos.py	Crea el dataset reproducible de 500 registros.
analisis_empleados.py	Carga, limpia, analiza, exporta y grafica.
requirements.txt	Declara las dependencias del proyecto.
salidas/resumen_empleados.xlsx	Excel con datos limpios, resumen y tabla dinámica.
salidas/graficos/	Seis figuras PNG listas para presentación.

4. Metodología del análisis

El flujo sigue las etapas clásicas de un análisis exploratorio: cargar, inspeccionar, limpiar, transformar, agrupar, exportar y visualizar. Cada etapa produce una salida verificable y se implementa en una función independiente.

Etapa	Técnica	Resultado
Carga	pd.read_excel(sheet_name='Empleados')	DataFrame original.
Inspección	shape, info, describe, isna	Diagnóstico de estructura y calidad.
Limpieza	Normalización de columnas y tipos	Datos consistentes.
Transformación	nombre_completo, pd.cut, cuantiles	Variables analíticas.
Análisis	groupby, agg, pivot_table, corr	Indicadores y tablas.
Exportación	pd.ExcelWriter con openpyxl	resumen_empleados.xlsx.
Visualización	matplotlib y seaborn	Seis figuras PNG.

Principio de diseño: el DataFrame original no se modifica. La limpieza se aplica sobre una copia y cada salida se escribe en la carpeta salidas, de modo que el análisis sea reproducible y seguro.

5. Código y explicación

5.1 Carga e inspección

La carga valida que el archivo exista y que las cuatro columnas obligatorias estén presentes. Después se revisan dimensiones, tipos, valores nulos y estadísticas descriptivas.

CÓDIGO

```
def cargar_datos(ruta: Path) -> pd.DataFrame:
    if not ruta.exists():
        raise FileNotFoundError(
            f"No se encontro el archivo {ruta.name}. Ejecute primero generar_datos.py."
        )
    dataframe = pd.read_excel(ruta, sheet_name="Empleados")
    faltantes = COLUMNAS_REQUERIDAS - set(dataframe.columns)
    if faltantes:
        raise ValueError(f"Faltan columnas obligatorias: {sorted(faltantes)}")
    return dataframe
```

SALIDA

```
Dimensiones: 500 filas x 4 columnas
Columnas: ['Nombre', 'Apellidos', 'Edad', 'Salario ($)']
Valores nulos: 0
```

Nombre	Apellidos	Edad	Salario ($)
Albert	Martinez Lopez	50	5,120
Albert	Chavez Salazar	32	2,590
Sandra	Rojas Medina	57	2,940
Fernando	Jimenez Vega	64	4,200
Monica	Castillo Ramos	49	3,900
Darius	Russell Harper	57	3,210
Roger	Tyler Best	36	3,560
Patricia	Chavez Salazar	61	3,710

5.2 Limpieza y transformación

Las columnas se normalizan a minúsculas y formato snake_case. Se convierten edad y salario a numérico, se eliminan duplicados y filas sin datos críticos, y se crean nombre_completo, grupo_edad y nivel_salarial a partir de cuantiles.

CÓDIGO

```
def limpiar_datos(dataframe: pd.DataFrame) -> pd.DataFrame:
    empleados = dataframe.copy()
    empleados.columns = (
        empleados.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_", regex=False)
        .str.replace("($)", "usd", regex=False)
        .str.replace("(", "", regex=False)
        .str.replace(")", "", regex=False)
    )

    for columna in ("nombre", "apellidos"):
        empleados[columna] = empleados[columna].astype("string").str.strip()

    empleados["edad"] = pd.to_numeric(empleados["edad"], errors="coerce")
    empleados["salario_usd"] = pd.to_numeric(empleados["salario_usd"], errors="coerce")
```

```
    empleados = (
        empleados
        .dropna(subset=["nombre", "apellidos", "edad", "salario_usd"])
        .drop_duplicates()
        .reset_index(drop=True)
    )
    empleados["edad"] = empleados["edad"].astype(int)
    empleados["salario_usd"] = empleados["salario_usd"].astype(float)

    empleados["nombre_completo"] = (
        empleados["nombre"] + " " + empleados["apellidos"]
    )
    empleados["grupo_edad"] = pd.cut(
        empleados["edad"],
        bins=[17, 29, 39, 49, 59, 70],
        labels=ETIQUETAS_EDAD,
    )

    q1 = empleados["salario_usd"].quantile(0.25)
    q3 = empleados["salario_usd"].quantile(0.75)
    empleados["nivel_salarial"] = empleados["salario_usd"].apply(
        lambda salario: "Bajo" if salario <= q1 else ("Alto" if salario >= q3 else "Medio")
    )
    return empleados
```

SALIDA

```
Columnas: ['nombre', 'apellidos', 'edad', 'salario_usd', 'nombre_completo', 'grupo_edad',
'nivel_salarial']
Registros útiles: 500
Grupos de edad: ['18-29', '30-39', '40-49', '50-59', '60-70']
Niveles salariales: ['Alto', 'Bajo', 'Medio']
```

5.3 Estadística descriptiva y agrupaciones

Se calculan los indicadores generales del salario y un resumen por grupo de edad con cantidad, promedio, mediana, mínimo, máximo y desviación estándar. La desviación permite comparar la dispersión entre grupos.

CÓDIGO

```
def resumen_general(empleados: pd.DataFrame) -> pd.Series:
    return empleados["salario_usd"].agg(
        ["count", "mean", "median", "min", "max", "std"]
    ).round(2)

def resumen_por_grupo_edad(empleados: pd.DataFrame) -> pd.DataFrame:
    return (
        empleados
        .groupby("grupo_edad", observed=True)
        .agg(
            cantidad=("salario_usd", "count"),
            salario_promedio=("salario_usd", "mean"),
            salario_mediano=("salario_usd", "median"),
            salario_minimo=("salario_usd", "min"),
            salario_maximo=("salario_usd", "max"),
            salario_desviacion=("salario_usd", "std"),
        )
        .round(2)
    )
```

RESUMEN GENERAL

estadistica	valor
count	500.00
mean	3,585.46
median	3,640.00
min	1,500.00
max	6,820.00
std	1,166.64

RESUMEN POR GRUPO DE EDAD

grupo_edad	cantidad	salario_promedio	salario_mediano	salario_minimo	salario_maximo	salario_desviacion
18-29	117	2,496.75	2,290.00	1,500.00	4,800.00	816.63
30-39	84	3,025.95	2,970.00	1,500.00	4,620.00	814.88
40-49	95	3,582.74	3,670.00	1,580.00	5,630.00	887.57
50-59	101	4,175.15	4,050.00	2,590.00	6,340.00	807.26
60-70	103	4,702.72	4,770.00	2,150.00	6,820.00	878.96

5.4 Tabla dinámica

La tabla dinámica cruza grupo de edad contra nivel salarial y cuenta cuántos empleados hay en cada combinación. Es útil para detectar concentraciones por grupo.

CÓDIGO

```
def crear_tabla_dinamica(empleados: pd.DataFrame) -> pd.DataFrame:
    return pd.pivot_table(
        empleados,
        values="salario_usd",
        index="grupo_edad",
        columns="nivel_salarial",
        aggfunc="count",
        fill_value=0,
        observed=True,
    )
```

TABLA DINÁMICA

grupo_edad	Alto	Bajo	Medio
18-29	4	71	42
30-39	6	32	46
40-49	17	18	60
50-59	35	2	64
60-70	63	2	38

5.5 Exportación a Excel

El archivo de salida contiene tres hojas: Datos_limpios, Resumen_grupo_edad y Tabla_dinamica. Se ajustan los anchos de columna y se congelan los encabezados de la hoja principal para facilitar la revisión.

CÓDIGO

```
def exportar_resultados(
    empleados: pd.DataFrame,
    resumen: pd.DataFrame,
    tabla: pd.DataFrame,
    ruta: Path,
) -> None:
    ruta.parent.mkdir(parents=True, exist_ok=True)
    with pd.ExcelWriter(ruta, engine="openpyxl") as writer:
        empleados.to_excel(writer, sheet_name="Datos_limpios", index=False)
        resumen.to_excel(writer, sheet_name="Resumen_grupo_edad")
        tabla.to_excel(writer, sheet_name="Tabla_dinamica")
        for hoja in ("Datos_limpios", "Resumen_grupo_edad", "Tabla_dinamica"):
            ajustar_ancho_columnas(writer, hoja)
        writer.sheets["Datos_limpios"].freeze_panes = "A2"
```

Hoja	Contenido
Datos_limpios	500 empleados con variables calculadas.
Resumen_grupo_edad	Indicadores salariales por rango de edad.
Tabla_dinamica	Conteo por grupo de edad y nivel salarial.

5.6 Visualización

Las figuras se generan con seaborn sobre matplotlib. Se utiliza una paleta consistente, títulos descriptivos y etiquetas en español. La función generar_graficos produce las seis imágenes y el dashboard final.

EXTRACTO DEL CÓDIGO DE VISUALIZACIÓN

```
sns.histplot(data=empleados, x="salario_usd", bins=20, kde=True)
sns.regplot(data=empleados, x="edad", y="salario_usd")
sns.barplot(data=empleados, x="grupo_edad", y="salario_usd", estimator="mean")
sns.boxplot(data=empleados, x="grupo_edad", y="salario_usd")
sns.heatmap(empleados[["edad", "salario_usd"]].corr(), annot=True, cmap="coolwarm")
```

El código completo de generar_graficos se incluye en el Anexo A. Cada figura se guarda con dpi=200 y bbox_inches='tight' para conservar calidad y evitar recortes.

6. Resultados del análisis

6.1 Dimensiones y calidad de los datos

Indicador	Valor
Filas	500
Columnas	4
Valores nulos	0
Registros útiles	500
Columnas calculadas	3 (nombre_completo, grupo_edad, nivel_salarial)

ESTADÍSTICAS DESCRIPTIVAS

estadistica	edad	salario_usd
count	500.00	500.00
mean	44.15	3,585.46
std	15.28	1,166.64
min	18.00	1,500.00
25%	30.00	2,692.50
50%	45.00	3,640.00
75%	57.00	4,362.50
max	70.00	6,820.00

6.2 Resumen general del salario

estadistica	valor
count	500.00
mean	3,585.46
median	3,640.00
min	1,500.00
max	6,820.00
std	1,166.64

El salario promedio es $3,585.46 y la mediana es $3,640.00. La media supera ligeramente a la mediana, lo que sugiere una distribución con algunos salarios altos que empujan el promedio hacia arriba.

6.3 Salario por grupo de edad

grupo_edad	cantidad	salario_promedio	salario_mediano	salario_minimo	salario_maximo	salario_desviacion
18-29	117	2,496.75	2,290.00	1,500.00	4,800.00	816.63
30-39	84	3,025.95	2,970.00	1,500.00	4,620.00	814.88
40-49	95	3,582.74	3,670.00	1,580.00	5,630.00	887.57
50-59	101	4,175.15	4,050.00	2,590.00	6,340.00	807.26
60-70	103	4,702.72	4,770.00	2,150.00	6,820.00	878.96

El grupo 60-70 presenta el salario promedio más alto ($4,702.72). El grupo 40-49 muestra la mayor desviación estándar ($887.57), es decir, la mayor heterogeneidad salarial.

6.4 Tabla dinámica por nivel salarial

grupo_edad	Alto	Bajo	Medio
18-29	4	71	42
30-39	6	32	46
40-49	17	18	60
50-59	35	2	64
60-70	63	2	38

Los grupos de mayor edad concentran más empleados en el nivel Alto, mientras que los grupos jóvenes se concentran en el nivel Bajo. Esta lectura es coherente con la relación positiva entre edad y salario.

6.5 Correlación edad-salario

Variable	Edad	Salario USD
Edad	1.000	0.722
Salario USD	0.722	1.000

El coeficiente de Pearson es 0.722. Esto indica una relación lineal positiva alta: a mayor edad, tiende a observarse un mayor salario. Sin embargo, la correlación no implica causalidad y el dataset no controla experiencia, cargo, industria ni ubicación.

7. Visualizaciones

Las seis figuras se generaron con el script y se guardaron en salidas/graficos. Cada una responde a una pregunta distinta del análisis.

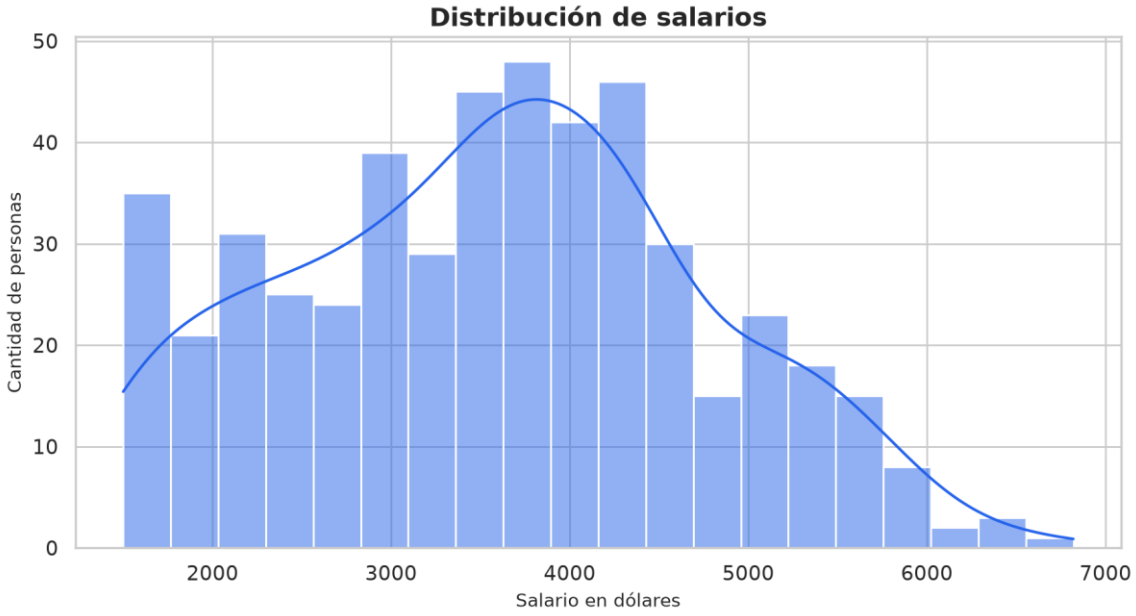


Figura 1. Distribución de salarios. Permite observar concentración, asimetría y valores extremos.

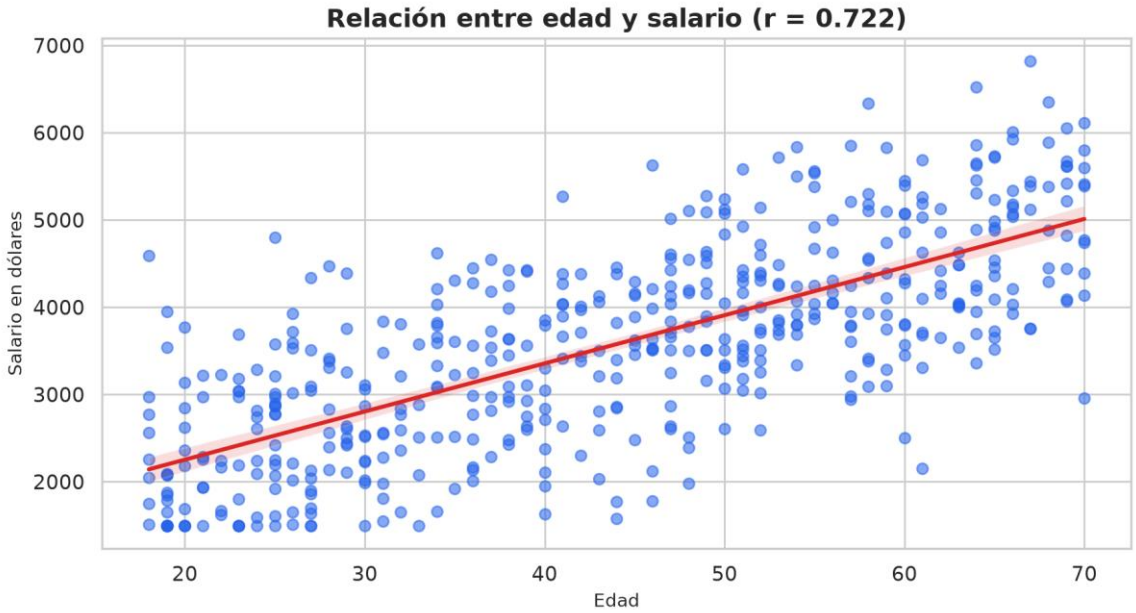


Figura 2. Relación entre edad y salario con línea de tendencia. La pendiente positiva acompaña la correlación.

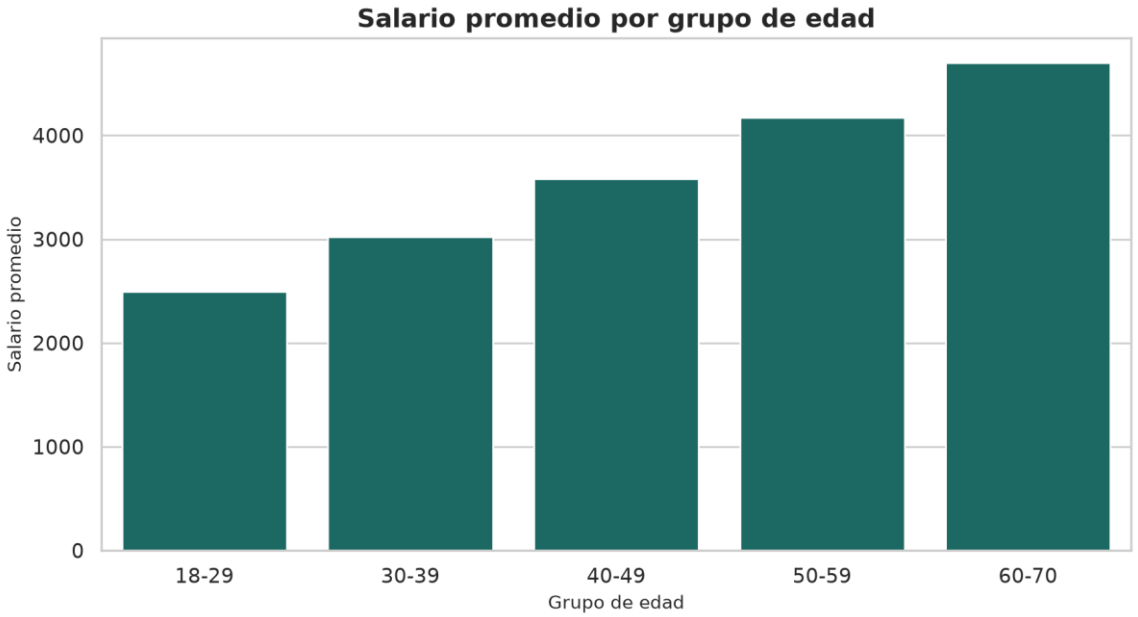


Figura 3. Salario promedio por grupo de edad. El grupo 60-70 lidera el promedio.

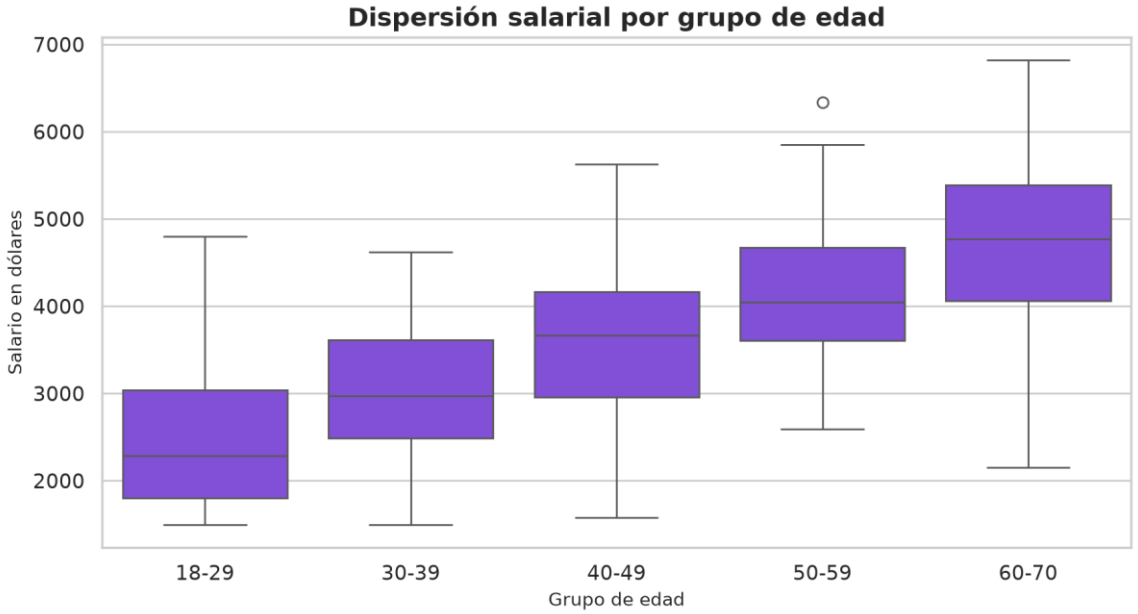


Figura 4. Dispersión salarial por grupo. La caja muestra el rango intercuartílico y los puntos, valores atípicos.

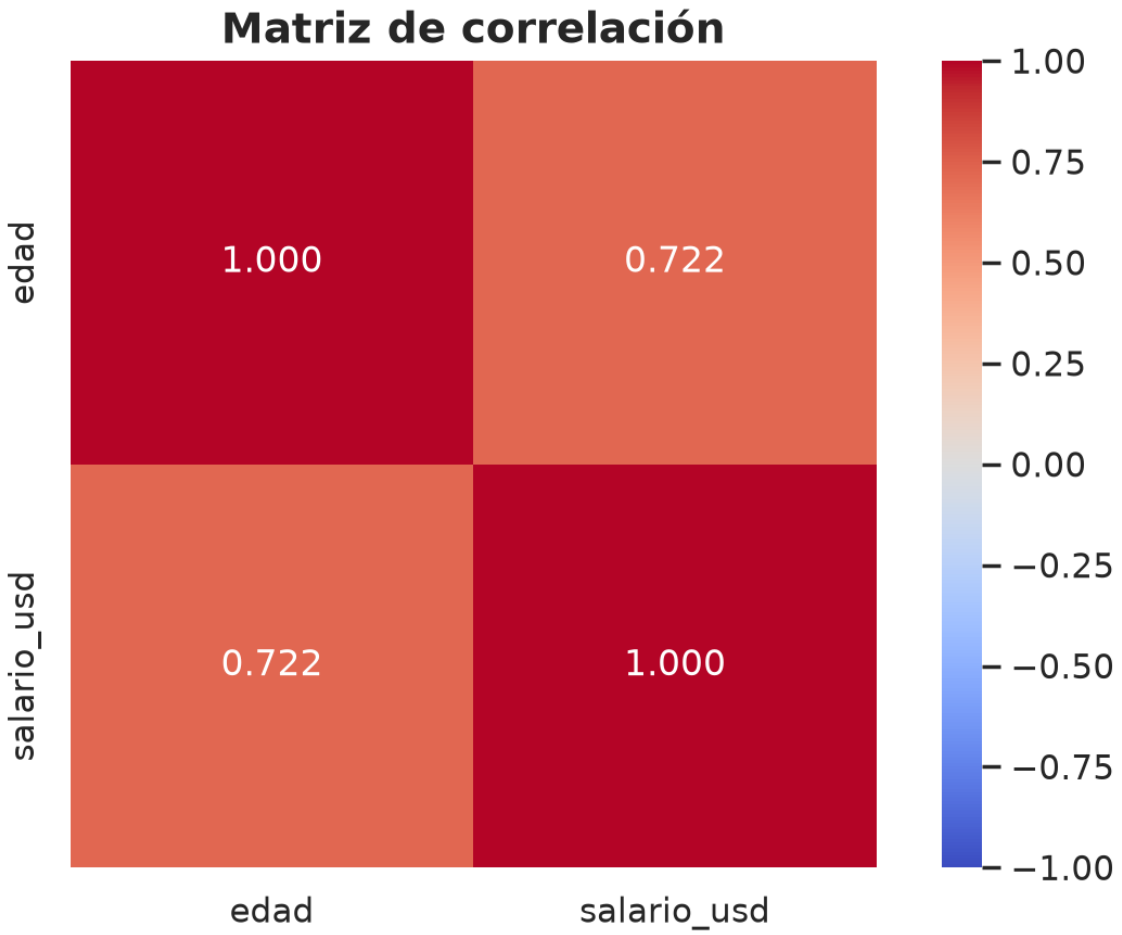


Figura 5. Matriz de correlación. El coeficiente entre edad y salario es 0.722.



Figura 6. Dashboard exploratorio con cuatro vistas complementarias.

8. Conclusiones y respuestas

8.1 Respuestas a las preguntas de análisis

Pregunta	Respuesta
¿Cuál es la edad promedio?	44.1 años.
¿Cuál es el salario promedio y la mediana?	Promedio $3,585.46; mediana $3,640.00.
¿Por qué la media y la mediana pueden diferir?	La media es sensible a valores extremos; la mediana ocupa la posición central.
¿Qué grupo de edad tiene el salario promedio más alto?	60-70, con $4,702.72.
¿Qué grupo presenta mayor dispersión salarial?	40-49, con desviación estándar de $887.57.
¿La edad parece explicar el salario?	Parcialmente. La correlación es 0.722; hay una relación positiva alta, pero no es una explicación completa.
¿Qué gráfico usaría para presentar a la gerencia?	El dashboard y el gráfico de barras por grupo de edad resumen lo esencial.
¿Qué limitaciones tiene el dataset?	Es ficticio y no incluye experiencia, cargo, industria, ubicación ni antigüedad.

8.2 Conclusión ejecutiva

Se analizaron 500 empleados con una edad promedio de 44.1 años. El salario promedio es $3,585.46 y la mediana $3,640.00. El grupo 60-70 concentra el mayor salario promedio y la correlación edad-salario es 0.722, una relación positiva alta. La edad aporta información, pero no determina por sí sola el salario: experiencia, cargo y sector son factores ausentes. Los resultados son exploratorios y deben interpretarse como una guía para nuevas preguntas, no como una conclusión causal.

Recomendación: complementar el dataset con variables de experiencia, cargo y antigüedad antes de usar los hallazgos para decisiones de compensación.

9. Ejecución en Visual Studio y VS Code

La carpeta puede abrirse como proyecto en Visual Studio o VS Code. El entorno virtual aísla las dependencias y el script genera el Excel y los gráficos en la carpeta salidas.

1. Abrir la carpeta 04_Pandas_Excel_Graficos en Visual Studio o VS Code.
2. Crear el entorno virtual con py -m venv .venv.
3. Activar el entorno con .\.venv\Scripts\Activate.ps1 en PowerShell.
4. Instalar dependencias con py -m pip install -r requirements.txt.
5. Generar el dataset con py generar_datos.py.
6. Ejecutar el análisis con py analisis_empleados.py.
7. Revisar salidas/resumen_empleados.xlsx y salidas/graficos/.

Elemento	Detalle
Dataset	datos_ficticios.xlsx
Generador	py generar_datos.py
Análisis	py analisis_empleados.py
Excel de salida	salidas/resumen_empleados.xlsx
Gráficos	salidas/graficos/
Dependencias	requirements.txt

En Visual Studio, seleccionar el intérprete del entorno virtual y presionar F5 sobre analisis_empleados.py. En VS Code, usar Run Python File. El análisis imprime cada etapa en la consola y no requiere entrada del usuario.

Anexo A. Código completo

Se incluyen los dos programas completos, sin comentarios ni marcas de edición. El generador produce el dataset y el análisis contiene todo el pipeline.

generar_datos.py

```
from pathlib import Path
import random

import pandas as pd

NOMBRES = [
    "Amber", "George", "Roger", "Albert", "Pamela", "Dylan", "Melissa", "Darius",
    "Laura", "Carlos", "Sofia", "Miguel", "Valeria", "Andres", "Camila", "Javier",
    "Daniela", "Fernando", "Gabriela", "Hector", "Isabel", "Jorge", "Karla", "Luis",
    "Monica", "Nicolas", "Patricia", "Ricardo", "Sandra", "Tomas",
]

APELLIDOS = [
    "Russell Harper", "Wright Marshall", "Henry Jacobs", "Tyler Best", "Watson Delacruz",
    "Wells White", "Burton Howell", "Lewis Gregory", "Ramirez Torres", "Gonzalez Ruiz",
    "Martinez Lopez", "Hernandez Diaz", "Perez Moreno", "Sanchez Romero", "Flores Vargas",
    "Rivera Castro", "Gomez Ortiz", "Diaz Munoz", "Vasquez Rojas", "Rojas Medina",
    "Silva Navarro", "Mendoza Reyes", "Cruz Aguilar", "Ortiz Pena", "Castillo Ramos",
    "Morales Herrera", "Jimenez Vega", "Ruiz Campos", "Chavez Salazar", "Guerrero Luna",
]

COLUMNAS = ["Nombre", "Apellidos", "Edad", "Salario ($)"]
CANTIDAD_REGISTROS = 500
SEMILLA = 2026


def generar_registros(cantidad: int, semilla: int) -> list[dict[str, object]]:
    generador = random.Random(semilla)
    registros = []
    for _ in range(cantidad):
        nombre = generador.choice(NOMBRES)
        apellidos = generador.choice(APELLIDOS)
        edad = generador.randint(18, 70)
        salario_base = 1200 + edad * 55 + generador.gauss(0, 850)
        salario = max(1500, min(9500, round(salario_base / 10) * 10))
        registros.append(
            {
                "Nombre": nombre,
                "Apellidos": apellidos,
                "Edad": edad,
                "Salario ($)": salario,
            }
        )
    return registros


def generar_dataframe(cantidad: int = CANTIDAD_REGISTROS, semilla: int = SEMILLA) -> pd.DataFrame:
    return pd.DataFrame(generar_registros(cantidad, semilla), columns=COLUMNAS)


def guardar_excel(dataframe: pd.DataFrame, ruta: Path) -> None:
    ruta.parent.mkdir(parents=True, exist_ok=True)
    with pd.ExcelWriter(ruta, engine="openpyxl") as writer:
        dataframe.to_excel(writer, sheet_name="Empleados", index=False)
        hoja = writer.sheets["Empleados"]
        for columna in hoja.columns:
            longitud = max(
                len(str(celda.value)) if celda.value is not None else 0
                for celda in columna
            )
            hoja.column_dimensions[columna[0].column_letter].width = min(
                max(longitud + 2, 12), 32
            )


def main() -> None:
    ruta = Path(__file__).with_name("datos_ficticios.xlsx")
    dataframe = generar_dataframe()
    guardar_excel(dataframe, ruta)
    print(f"Archivo generado: {ruta.name}")
    print(f"Registros: {len(dataframe)}")
    print(dataframe.head(8).to_string(index=False))


if __name__ == "__main__":
```

```
    main()
```

analisis_empleados.py

```
from pathlib import Path

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

BASE_DIR = Path(__file__).resolve().parent
ARCHIVO_DATOS = BASE_DIR / "datos_ficticios.xlsx"
CARPETA_SALIDAS = BASE_DIR / "salidas"
CARPETA_GRAFICOS = CARPETA_SALIDAS / "graficos"
COLUMNAS_REQUERIDAS = {"Nombre", "Apellidos", "Edad", "Salario ($)"}
ETIQUETAS_EDAD = ["18-29", "30-39", "40-49", "50-59", "60-70"]


def cargar_datos(ruta: Path) -> pd.DataFrame:
    if not ruta.exists():
        raise FileNotFoundError(
            f"No se encontro el archivo {ruta.name}. Ejecute primero generar_datos.py."
        )
    dataframe = pd.read_excel(ruta, sheet_name="Empleados")
    faltantes = COLUMNAS_REQUERIDAS - set(dataframe.columns)
    if faltantes:
        raise ValueError(f"Faltan columnas obligatorias: {sorted(faltantes)}")
    return dataframe


def limpiar_datos(dataframe: pd.DataFrame) -> pd.DataFrame:
    empleados = dataframe.copy()
    empleados.columns = (
        empleados.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_", regex=False)
        .str.replace("($)", "usd", regex=False)
        .str.replace("(", "", regex=False)
        .str.replace(")", "", regex=False)
    )

    for columna in ("nombre", "apellidos"):
        empleados[columna] = empleados[columna].astype("string").str.strip()

    empleados["edad"] = pd.to_numeric(empleados["edad"], errors="coerce")
    empleados["salario_usd"] = pd.to_numeric(empleados["salario_usd"], errors="coerce")
    empleados = (
        empleados
        .dropna(subset=["nombre", "apellidos", "edad", "salario_usd"])
        .drop_duplicates()
        .reset_index(drop=True)
    )
    empleados["edad"] = empleados["edad"].astype(int)
    empleados["salario_usd"] = empleados["salario_usd"].astype(float)

    empleados["nombre_completo"] = (
        empleados["nombre"] + " " + empleados["apellidos"]
    )
    empleados["grupo_edad"] = pd.cut(
        empleados["edad"],
        bins=[17, 29, 39, 49, 59, 70],
        labels=ETIQUETAS_EDAD,
    )

    q1 = empleados["salario_usd"].quantile(0.25)
    q3 = empleados["salario_usd"].quantile(0.75)
    empleados["nivel_salarial"] = empleados["salario_usd"].apply(
        lambda salario: "Bajo" if salario <= q1 else ("Alto" if salario >= q3 else "Medio")
    )
    return empleados


def resumen_general(empleados: pd.DataFrame) -> pd.Series:
    return empleados["salario_usd"].agg(
        ["count", "mean", "median", "min", "max", "std"]
    ).round(2)


def resumen_por_grupo_edad(empleados: pd.DataFrame) -> pd.DataFrame:
    return (
        empleados
```

```
        .groupby("grupo_edad", observed=True)
        .agg(
            cantidad=("salario_usd", "count"),
            salario_promedio=("salario_usd", "mean"),
            salario_mediano=("salario_usd", "median"),
            salario_minimo=("salario_usd", "min"),
            salario_maximo=("salario_usd", "max"),
            salario_desviacion=("salario_usd", "std"),
        )
        .round(2)
    )


def crear_tabla_dinamica(empleados: pd.DataFrame) -> pd.DataFrame:
    return pd.pivot_table(
        empleados,
        values="salario_usd",
        index="grupo_edad",
        columns="nivel_salarial",
        aggfunc="count",
        fill_value=0,
        observed=True,
    )


def ajustar_ancho_columnas(writer: pd.ExcelWriter, nombre_hoja: str) -> None:
    hoja = writer.sheets[nombre_hoja]
    for columna in hoja.columns:
        longitud = max(
            len(str(celda.value)) if celda.value is not None else 0
            for celda in columna
        )
        hoja.column_dimensions[columna[0].column_letter].width = min(
            max(longitud + 2, 12), 34
        )


def exportar_resultados(
    empleados: pd.DataFrame,
    resumen: pd.DataFrame,
    tabla: pd.DataFrame,
    ruta: Path,
) -> None:
    ruta.parent.mkdir(parents=True, exist_ok=True)
    with pd.ExcelWriter(ruta, engine="openpyxl") as writer:
        empleados.to_excel(writer, sheet_name="Datos_limpios", index=False)
        resumen.to_excel(writer, sheet_name="Resumen_grupo_edad")
        tabla.to_excel(writer, sheet_name="Tabla_dinamica")
        for hoja in ("Datos_limpios", "Resumen_grupo_edad", "Tabla_dinamica"):
            ajustar_ancho_columnas(writer, hoja)
        writer.sheets["Datos_limpios"].freeze_panes = "A2"


def configurar_estilo() -> None:
    sns.set_theme(style="whitegrid", palette="deep")
    plt.rcParams.update(
        {
            "font.family": "DejaVu Sans",
            "axes.titlesize": 14,
            "axes.titleweight": "bold",
            "axes.labelsize": 10,
            "figure.dpi": 110,
        }
    )


def guardar_figura(figura: plt.Figure, ruta: Path) -> Path:
    figura.savefig(ruta, dpi=200, bbox_inches="tight")
    plt.close(figura)
    return ruta


def generar_graficos(empleados: pd.DataFrame, carpeta: Path) -> dict[str, Path]:
    carpeta.mkdir(parents=True, exist_ok=True)
    configurar_estilo()
    rutas = {}

    figura, eje = plt.subplots(figsize=(9, 5))
    sns.histplot(
        data=empleados,
        x="salario_usd",
        bins=20,
        kde=True,
        color="#2563EB",
        ax=eje,
    )
    eje.set_title("Distribución de salarios")
    eje.set_xlabel("Salario en dólares")
```

```
    eje.set_ylabel("Cantidad de personas")
    figura.tight_layout()
    rutas["histograma"] = guardar_figura(
        figura, carpeta / "histograma_salarios.png"
    )

    correlacion = empleados["edad"].corr(empleados["salario_usd"])
    figura, eje = plt.subplots(figsize=(9, 5))
    sns.regplot(
        data=empleados,
        x="edad",
        y="salario_usd",
        scatter_kws={"alpha": 0.55, "color": "#2563EB"},
        line_kws={"color": "#DC2626"},
        ax=eje,
    )
    eje.set_title(f"Relación entre edad y salario (r = {correlacion:.3f})")
    eje.set_xlabel("Edad")
    eje.set_ylabel("Salario en dólares")
    figura.tight_layout()
    rutas["dispersion"] = guardar_figura(
        figura, carpeta / "edad_vs_salario.png"
    )

    figura, eje = plt.subplots(figsize=(9, 5))
    sns.barplot(
        data=empleados,
        x="grupo_edad",
        y="salario_usd",
        estimator="mean",
        errorbar=None,
        color="#0F766E",
        ax=eje,
    )
    eje.set_title("Salario promedio por grupo de edad")
    eje.set_xlabel("Grupo de edad")
    eje.set_ylabel("Salario promedio")
    figura.tight_layout()
    rutas["barras"] = guardar_figura(
        figura, carpeta / "promedio_grupo_edad.png"
    )

    figura, eje = plt.subplots(figsize=(9, 5))
    sns.boxplot(
        data=empleados,
        x="grupo_edad",
        y="salario_usd",
        color="#7C3AED",
        ax=eje,
    )
    eje.set_title("Dispersión salarial por grupo de edad")
    eje.set_xlabel("Grupo de edad")
    eje.set_ylabel("Salario en dólares")
    figura.tight_layout()
    rutas["boxplot"] = guardar_figura(
        figura, carpeta / "boxplot_salarios.png"
    )

    correlacion_numerica = empleados[["edad", "salario_usd"]].corr()
    figura, eje = plt.subplots(figsize=(6.4, 4.6))
    sns.heatmap(
        correlacion_numerica,
        annot=True,
        fmt=".3f",
        cmap="coolwarm",
        vmin=-1,
        vmax=1,
        square=True,
        ax=eje,
    )
    eje.set_title("Matriz de correlación")
    figura.tight_layout()
    rutas["correlacion"] = guardar_figura(
        figura, carpeta / "correlacion.png"
    )

    figura, ejes = plt.subplots(2, 2, figsize=(14, 9))
    sns.histplot(
        data=empleados,
        x="salario_usd",
        bins=20,
        kde=True,
        color="#2563EB",
        ax=ejes[0, 0],
    )
    ejes[0, 0].set_title("Distribución de salarios")
    ejes[0, 0].set_xlabel("Salario en dólares")
    ejes[0, 0].set_ylabel("Cantidad")
```

```
    sns.boxplot(
        data=empleados,
        x="grupo_edad",
        y="salario_usd",
        color="#7C3AED",
        ax=ejes[0, 1],
    )
    ejes[0, 1].set_title("Salarios por grupo de edad")
    ejes[0, 1].set_xlabel("Grupo de edad")
    ejes[0, 1].set_ylabel("Salario en dólares")

    sns.scatterplot(
        data=empleados,
        x="edad",
        y="salario_usd",
        hue="nivel_salarial",
        hue_order=["Bajo", "Medio", "Alto"],
        palette={"Bajo": "#DC2626", "Medio": "#F59E0B", "Alto": "#16A34A"},
        alpha=0.75,
        ax=ejes[1, 0],
    )
    ejes[1, 0].set_title("Edad vs salario por nivel salarial")
    ejes[1, 0].set_xlabel("Edad")
    ejes[1, 0].set_ylabel("Salario en dólares")

    conteo_nivel = (
        empleados["nivel_salarial"]
        .value_counts()
        .reindex(["Bajo", "Medio", "Alto"])
    )
    sns.barplot(
        x=conteo_nivel.index,
        y=conteo_nivel.values,
        hue=conteo_nivel.index,
        palette={"Bajo": "#DC2626", "Medio": "#F59E0B", "Alto": "#16A34A"},
        legend=False,
        ax=ejes[1, 1],
    )
    ejes[1, 1].set_title("Cantidad por nivel salarial")
    ejes[1, 1].set_xlabel("Nivel salarial")
    ejes[1, 1].set_ylabel("Cantidad")

    figura.suptitle("Dashboard exploratorio de empleados", fontsize=16, fontweight="bold")
    figura.tight_layout()
    rutas["dashboard"] = guardar_figura(
        figura, carpeta / "dashboard_empleados.png"
    )
    return rutas


def main() -> None:
    print("=" * 74)
    print("ANALISIS EXPLORATORIO DE EMPLEADOS CON PANDAS")
    print("=" * 74)

    if not ARCHIVO_DATOS.exists():
        from generar_datos import generar_dataframe, guardar_excel

        guardar_excel(generar_dataframe(), ARCHIVO_DATOS)
        print(f"Dataset generado automaticamente: {ARCHIVO_DATOS.name}")

    dataframe = cargar_datos(ARCHIVO_DATOS)
    print("\n[1] Carga y vista previa del archivo Excel")
    print(f"Dimensiones: {dataframe.shape[0]} filas x {dataframe.shape[1]} columnas")
    print(dataframe.head(8).to_string(index=False))

    print("\n[2] Información general del DataFrame")
    dataframe.info()

    print("\n[3] Valores faltantes por columna")
    print(dataframe.isna().sum().to_string())

    empleados = limpiar_datos(dataframe)
    print("\n[4] Limpieza y columnas calculadas")
    print(f"Columnas normalizadas: {empleados.columns.tolist()}")
    print(f"Registros útiles: {len(empleados)}")

    print("\n[5] Estadísticas descriptivas de variables numéricas")
    print(empleados[["edad", "salario_usd"]].describe().round(2).to_string())

    print("\n[6] Resumen general del salario")
    print(resumen_general(empleados).to_string())

    resumen_grupo = resumen_por_grupo_edad(empleados)
    print("\n[7] Resumen salarial por grupo de edad")
    print(resumen_grupo.to_string(line_width=110))
```

```
    tabla = crear_tabla_dinamica(empleados)
    print("\n[8] Tabla dinámica: grupo de edad por nivel salarial")
    print(tabla.to_string())

    correlacion = empleados["edad"].corr(empleados["salario_usd"])
    print("\n[9] Analisis de Relación entre edad y salario")
    print(f"Coeficiente de correlación de Pearson: {correlacion:.3f}")

    mejor_grupo = resumen_grupo["salario_promedio"].idxmax()
    grupo_disperso = resumen_grupo["salario_desviacion"].idxmax()
    print("\n[10] Respuestas a las preguntas de análisis")
    print(f"Edad promedio: {empleados['edad'].mean():.1f} años")
    print(f"Salario promedio: ${empleados['salario_usd'].mean():,.2f}")
    print(f"Salario mediano: ${empleados['salario_usd'].median():,.2f}")
    print(
        f"Grupo con mayor salario promedio: {mejor_grupo} "
        f"(${resumen_grupo.loc[mejor_grupo, 'salario_promedio']:,.2f})"
    )
    print(
        f"Grupo con mayor dispersión salarial: {grupo_disperso} "
        f"(desviación estándar ${resumen_grupo.loc[grupo_disperso, 'salario_desviacion']:,.2f})"
    )
    print(
        "La media y la mediana pueden diferir porque la media es sensible a "
        "salarios extremos, mientras que la mediana representa el valor central."
    )
    print(
        f"La edad explica parcialmente el salario: la correlación es {correlacion:.3f}, "
        "por lo que existen otros factores no incluidos en el dataset."
    )

    ruta_excel = CARPETA_SALIDAS / "resumen_empleados.xlsx"
    exportar_resultados(empleados, resumen_grupo, tabla, ruta_excel)
    print(f"\n[11] Resultados exportados: {ruta_excel.relative_to(BASE_DIR)}")

    rutas = generar_graficos(empleados, CARPETA_GRAFICOS)
    print("\n[12] Gráficos generados")
    for nombre, ruta in rutas.items():
        print(f"{nombre}: {ruta.relative_to(BASE_DIR)}")

    print("\n[13] Conclusión ejecutiva")
    print(
        f"Se analizaron {len(empleados)} empleados con una edad promedio de "
        f"{empleados['edad'].mean():.1f} años. El salario promedio es de "
        f"${empleados['salario_usd'].mean():,.2f} y la mediana de "
        f"${empleados['salario_usd'].median():,.2f}. El grupo {mejor_grupo} "
        "concentra el mayor salario promedio, mientras que la correlación entre "
        f"edad y salario es {correlacion:.3f}. Esto sugiere una relación positiva "
        "alta: la edad aporta información, pero no determina por sí sola el "
        "salario. El dataset es ficticio y no incorpora experiencia, cargo, "
        "ubicación ni antigüedad, por lo que sus hallazgos son exploratorios."
    )


if __name__ == "__main__":
    main()
```

Anexo B. Salida completa de consola

La siguiente es la salida completa de analisis_empleados.py. Sirve como evidencia de ejecución y permite verificar cada indicador presentado en el documento.

```
======================================================================
ANALISIS EXPLORATORIO DE EMPLEADOS CON PANDAS
======================================================================

[1] Carga y vista previa del archivo Excel
Dimensiones: 500 filas x 4 columnas
  Nombre      Apellidos  Edad  Salario ($)
  Albert Martinez Lopez    50         5120
  Albert Chavez Salazar    32         2590
  Sandra   Rojas Medina    57         2940
Fernando   Jimenez Vega    64         4200
  Monica Castillo Ramos    49         3900
  Darius Russell Harper    57         3210
   Roger     Tyler Best    36         3560
Patricia Chavez Salazar    61         3710

[2] Información general del DataFrame
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 4 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Nombre       500 non-null    str
 1   Apellidos    500 non-null    str
 2   Edad         500 non-null    int64
 3   Salario ($)  500 non-null    int64
dtypes: int64(2), str(2)
memory usage: 15.8 KB

[3] Valores faltantes por columna
Nombre         0
Apellidos      0
Edad           0
Salario ($)    0

[4] Limpieza y columnas calculadas
Columnas normalizadas: ['nombre', 'apellidos', 'edad', 'salario_usd', 'nombre_completo', 'grupo_edad',
'nivel_salarial']
Registros útiles: 500

[5] Estadísticas descriptivas de variables numéricas
         edad  salario_usd
count  500.00       500.00
mean    44.15      3585.46
std     15.28      1166.64
min     18.00      1500.00
25%     30.00      2692.50
50%     45.00      3640.00
75%     57.00      4362.50
max     70.00      6820.00

[6] Resumen general del salario
count      500.00
mean      3585.46
median    3640.00
min       1500.00
max       6820.00
std       1166.64

[7] Resumen salarial por grupo de edad
            cantidad  salario_promedio  salario_mediano  salario_minimo  salario_maximo  salario_desviacion
grupo_edad
18-29            117           2496.75           2290.0          1500.0          4800.0              816.63
30-39             84           3025.95           2970.0          1500.0          4620.0              814.88
40-49             95           3582.74           3670.0          1580.0          5630.0              887.57
50-59            101           4175.15           4050.0          2590.0          6340.0              807.26
60-70            103           4702.72           4770.0          2150.0          6820.0              878.96

[8] Tabla dinámica: grupo de edad por nivel salarial
nivel_salarial  Alto  Bajo  Medio
grupo_edad
18-29              4    71     42
30-39              6    32     46
40-49             17    18     60
50-59             35     2     64
60-70             63     2     38

[9] Analisis de Relación entre edad y salario
```

```
Coeficiente de correlación de Pearson: 0.722

[10] Respuestas a las preguntas de análisis
Edad promedio: 44.1 años
Salario promedio: $3,585.46
Salario mediano: $3,640.00
Grupo con mayor salario promedio: 60-70 ($4,702.72)
Grupo con mayor dispersión salarial: 40-49 (desviación estándar $887.57)
La media y la mediana pueden diferir porque la media es sensible a salarios extremos, mientras que la mediana
representa el valor central.
La edad explica parcialmente el salario: la correlación es 0.722, por lo que existen otros factores no incluidos en el
dataset.

[11] Resultados exportados: salidas\resumen_empleados.xlsx

[12] Gráficos generados
histograma: salidas\graficos\histograma_salarios.png
dispersion: salidas\graficos\edad_vs_salario.png
barras: salidas\graficos\promedio_grupo_edad.png
boxplot: salidas\graficos\boxplot_salarios.png
correlacion: salidas\graficos\correlacion.png
dashboard: salidas\graficos\dashboard_empleados.png

[13] Conclusión ejecutiva
Se analizaron 500 empleados con una edad promedio de 44.1 años. El salario promedio es de $3,585.46 y la mediana de
$3,640.00. El grupo 60-70 concentra el mayor salario promedio, mientras que la correlación entre edad y salario es
0.722. Esto sugiere una relación positiva alta: la edad aporta información, pero no determina por sí sola el salario.
El dataset es ficticio y no incorpora experiencia, cargo, ubicación ni antigüedad, por lo que sus hallazgos son
exploratorios.
```